GTM SYSTEMS DELIVERY PLAN

SALDEV-1500 Implementation Plan

Gearset, Copado and MuleSoft CI/CD rollout

Plan detail	Value
Prepared for	Flywire GTM Systems, Salesforce, Integration and DevOps teams
Prepared on	2 September 2026
Status	Draft for stakeholder review and implementation approval
Delivery horizon	12-week controlled rollout, subject to access and licensing
Source ticket	SALDEV-1500

Implementation decision
Use Gearset for Salesforce pull-request validation, metadata analysis and lower-environment delivery; use Copado for governed Salesforce release promotion, approvals and production deployment; use GitHub Actions and Anypoint for MuleSoft build, MUnit testing, packaging and runtime promotion. GitHub remains the shared source of truth and Jira remains the work and evidence record.

Important control
There will be one production deployment owner per platform. Copado is the only Salesforce production deployer. The protected MuleSoft pipeline is the only Anypoint production deployer. Gearset production jobs remain validation-only. This prevents two tools from deploying different versions of the same change.

This is a separate implementation plan. It does not replace the SALDEV-1500 operating model and does not make changes to Jira, GitHub, Gearset, Copado, MuleSoft or Salesforce by itself.

Design note: the document uses only native Word text, tables and styles. No Figma design work, exports or Figma assets are included.

1. What this plan is meant to achieve

The goal is not to install three tools and call the program complete. The goal is to give delivery teams one understandable route from a Jira story to production, even when a release includes both Salesforce metadata and a MuleSoft integration.

A developer should know where to commit, which checks will run, who can approve a promotion and where to find the final evidence. A release manager should be able to identify the exact Salesforce commit, MuleSoft artifact version and deployment jobs without reconstructing the release from chat messages.

Outcomes

- A controlled Git-based delivery path for Salesforce and MuleSoft work.
- Automated Salesforce comparison, dependency analysis, validation and QA deployment through Gearset.
- Copado-managed Salesforce release packaging, approval, UAT and production promotion.
- MuleSoft CI using Maven and MUnit, with versioned assets published to Exchange and deployed through protected Anypoint environments.
- A Jira board whose status reflects real evidence rather than optimistic manual updates.
- One cross-platform release manifest linking Jira, Git commits, Gearset jobs, Copado promotions and MuleSoft artifacts.
- Documented rollback, back-promotion, support ownership and operating metrics.

Success looks like this

Pilot definition
One low-risk release moves through Salesforce and MuleSoft development, automated validation, QA, UAT and protected production without direct changes, scope substitution or missing evidence. A deliberate test failure is blocked, corrected and re-run before the pilot is accepted.

2. Scope, boundaries and assumptions

In scope

- Jira workflow, evidence fields, transition ownership and board views for SALDEV delivery.
- GitHub repository structure, branch protection, pull-request templates and release manifest.
- Gearset team connections, metadata filters, PR validation, QA CI jobs and production validation-only job.

- Copado governance/training environment, Salesforce pipeline, user-story/release configuration, approvals, promotions and back-promotion.
- MuleSoft repository standards, MUnit quality gates, Exchange publishing and Anypoint deployment jobs.
- Cross-platform test strategy, security controls, observability, rollback and handover.

Not included in this document

- Procurement approval or a statement that current Gearset, Copado or MuleSoft licenses are sufficient.
- A decision on CloudHub 2.0, Runtime Fabric or on-premises Mule runtime; the current target must be confirmed during discovery.
- Bulk migration of every existing repository or pipeline before the pilot proves the approach.
- Direct production changes, credential creation, package installation or tool configuration.
- Figma prototypes, diagrams, exports or design-system work.

Assumptions to validate in week 1

Assumption	Why it matters	Owner
GitHub Enterprise is available and approved	It is the common source of truth and branch-control layer.	Platform/DevOps
Gearset licenses include required CI and team-shared capabilities	The design uses shared Git-to-org jobs and PR validation.	Tool owner
Copado edition and pipeline mode are confirmed	Source Format and Metadata Pipelines have different capabilities and limitations.	Copado owner
A Copado governance or training environment is available	The team needs a safe place to prove pipeline configuration before live use.	Salesforce platform
Anypoint runtime target and business group are known	Packaging, credentials and deployment settings depend on the target.	Integration lead
Jira administrators can create fields, statuses and automation	The evidence workflow cannot be implemented without project-level control.	Jira owner

3. Tool responsibilities

The tools overlap in places. The plan makes the boundaries explicit so a release cannot be promoted independently by two systems.

Platform	Primary responsibility	What it must not become
Jira	Scope, acceptance criteria, ownership, risk, approvals, status and final evidence	A replacement for Git history or deployment logs
GitHub	Canonical source, branch protection, pull requests, immutable commit references and release manifest	A place for unreviewed environment-specific secrets
Gearset	Salesforce metadata compare, dependency analysis, PR validation, QA delivery and production validation	A second Salesforce production deployment path
Copado	Salesforce release orchestration, promotion bundle, UAT/production approval, deployment and back-promotion	A parallel source of truth disconnected from GitHub
GitHub Actions	Repository-native checks and MuleSoft CI/build automation	An unapproved production bypass
MuleSoft Anypoint	API assets, Mule runtime deployment, environment configuration, policies, logs and monitoring	A source repository for unreviewed changes
Confluence	Published runbooks, standards, training and operational guidance	The authoritative status of a live release

Dual-tool Salesforce rule

Gearset + Copado
Gearset owns CI and technical validation. Copado owns controlled release promotion. Gearset may deploy automatically to Integration and QA, but its UAT and Production jobs remain disabled or validation-only. Copado deploys the exact approved Git commit to UAT and Production. Every Copado promotion records the corresponding Gearset validation job.

MuleSoft release rule

Copado + Anypoint
Copado coordinates the release and approval record; the protected MuleSoft pipeline performs the Anypoint deployment. A direct Copado-to-Anypoint trigger is optional and must not be assumed until the selected Copado pipeline supports the integration securely. The default pilot uses paired approvals and shared release evidence rather than an unproven custom callout.

4. Target delivery architecture

The delivery chain is deliberately simple enough to explain in one line:

End-to-end path
Jira story -> GitHub branch and pull request -> Gearset Salesforce checks + MuleSoft GitHub Actions checks -> approved merge -> Gearset QA + MuleSoft QA deployment -> integrated test -> Copado release approval -> Salesforce and MuleSoft protected production deployment -> Jira evidence and closure

Repository model

Repository	Contents	Required controls
gtm-salesforce	Salesforce DX source, manifests, Apex/LWC tests, permission sets and deployment notes	CODEOWNERS, PR validation, protected integration/uat/main branches
gtm-mulesoft-integrations	Mule applications, RAML/OAS, DataWeave, MUnit tests, POM files and properties templates	MUnit and package checks, protected main, versioned release tags
gtm-release-config	Cross-platform release manifests, environment mappings and runbook references	Release-manager approval and immutable production tags

Release manifest

Each coordinated release gets one manifest. It may be a versioned YAML/JSON file or a controlled Jira release record, but it must capture the same identifiers every time.

- Jira release and included stories.
- Salesforce repository, commit SHA, Gearset validation job and Copado promotion/deployment ID.
- MuleSoft repository, commit SHA, packaged artifact version, Exchange asset version and Anypoint deployment job.
- Target environments, approvers, test results, manual steps, rollback point and final outcome.

Environment mapping

Stage	Salesforce path	MuleSoft path	Promotion owner
Developer	Personal sandbox or scratch org	Local/isolated development	Developer
Integration	Gearset deploys approved integration branch	Pipeline deploys snapshot to Mule Dev	Dev lead
QA	Gearset deploys approved QA candidate	Pipeline deploys versioned candidate to Mule QA	QA lead
UAT	Copado deploys approved release candidate	Protected pipeline promotes same artifact to Mule UAT	Business owner + release manager
Production	Copado deploys immutable approved commit	Protected pipeline deploys immutable artifact to Anypoint Prod	Release manager

5. Delivery process from story to production

1. Refine the Jira story. Confirm the business outcome, affected Salesforce components, MuleSoft APIs/flows, data and security impact, test personas, dependencies and rollback outline.

2. Create linked ticket branches. Use the same Jira key in the Salesforce and MuleSoft repositories when both are affected.
3. Build in isolation. Salesforce work is captured in DX source format; MuleSoft work includes the API contract, Mule configuration, DataWeave, POM changes and MUnit coverage.
4. Open focused pull requests. The PR lists added, changed and deleted components, manual steps, risk class and cross-repository dependencies.
5. Run CI. Gearset validates Salesforce metadata and tests against the target org. GitHub Actions runs Mule Maven packaging, MUnit, coverage and API-contract checks.
6. Review and merge only after both streams pass. The release manifest pins the approved Salesforce SHA and MuleSoft artifact version.
7. Deploy to Integration and QA. Gearset delivers Salesforce changes; the protected Mule pipeline deploys the packaged Mule artifact. Rebuilding a different artifact during promotion is not allowed.
8. Run integrated QA. Test the API contract, authentication, retries, error mapping, idempotency, data behavior, permissions and Salesforce user journeys.
9. Approve UAT and release. Copado groups the Salesforce stories and records the release approval. The paired MuleSoft artifact and Anypoint job are linked in the same release manifest.
10. Deploy through protected production paths. Copado deploys Salesforce; the MuleSoft production job deploys to Anypoint. Both use named service identities and retain immutable evidence.
11. Verify and close. Complete smoke checks, monitor logs and alerts, reconcile any manual actions, link known issues and write the evidence back to Jira before Done.

6. Twelve-week implementation roadmap

Phase	Weeks	Main work	Exit condition
0. Decide and discover	1-2	Licenses, current pipelines, runtime target, repositories, environments, security and owner confirmation	Signed design decisions and pilot scope
1. Foundation	2-4	GitHub controls, Jira workflow, service identities, baseline metadata and repository templates	Controlled repositories and ready backlog
2. Salesforce CI	4-6	Gearset connections, filters, PR validation, Integration/QA jobs and production validation-only job	Salesforce non-prod path proven
3. Governed Salesforce CD	6-8	Copado training pipeline, release/user-story model, UAT/Prod promotion, approvals and back-promotion	Salesforce release path proven
4. MuleSoft delivery	5-8	Maven/MUnit CI, Exchange publishing, Anypoint Dev/QA/UAT/Prod deployment and monitoring	Immutable Mule artifact path proven

Phase	Weeks	Main work	Exit condition
5. Integrated pilot	9-10	One low-risk cross-platform change, failure test, QA/UAT, rollback rehearsal and production smoke	Pilot accepted; critical gaps closed
6. Handover and scale	11-12	Training, runbooks, support model, metrics, backlog and adoption plan	BAU ownership accepted

Implementation sequence

Gearset and MuleSoft CI can be built in parallel after the repository and service-account foundations are ready. Copado production work starts only after the team has confirmed which pipeline mode is licensed and tested the intended features in a training pipeline.

7. Workstream plan

Workstream A - Governance and access

- Name the product owner, Salesforce lead, Gearset owner, Copado owner, MuleSoft lead, QA lead, security approver and release manager.
- Create service identities with least privilege and non-personal ownership. Record renewal and offboarding procedures.
- Approve repository access, connected applications, Anypoint connected app, IP allowlisting and secret storage.
- Document break-glass access, approval rules and monthly access review.

Workstream B - GitHub and Jira foundation

- Create or validate repositories, CODEOWNERS, branch naming and protected branches.
- Add PR templates for Salesforce, MuleSoft and coordinated releases.
- Configure Jira statuses, evidence fields, blocked/rework paths and release approval fields.
- Link Jira keys, branches, pull requests, Gearset jobs, Copado promotions and MuleSoft jobs.

Workstream C - Gearset Salesforce CI

- Create team-shared Git and org connections using approved service accounts.
- Create one reviewed metadata filter and explicit permission/destructive-change policy.
- Configure PR validation against the correct target org with Apex, LWC and static-analysis gates.
- Configure branch-triggered Integration and QA deployment jobs.

- Configure Production as validation-only and restrict deployment access.
- Store job history, component scope, test results and omissions as release evidence.

Workstream D - Copado Salesforce CD

- Confirm Source Format versus Metadata Pipelines against required capabilities and current licensing.
- Set up a governance/training environment before associating live releases.
- Configure environments, credentials, branch mapping, user-story/release model and approval owners.
- Build UAT and Production promotions from approved Git references only.
- Configure deployment steps, manual tasks, back-promotion and evidence retention.
- Prove failed promotion, rejected approval, rollback decision and back-promotion behavior.

Workstream E - MuleSoft CI/CD

- Confirm the Anypoint organization, business group, runtime plane, regions, environment names and deployment target.
- Standardize POM files, Mule Maven Plugin configuration, MUnit tests, coverage thresholds and properties templates.
- Run API specification validation, Maven packaging, MUnit and security/dependency checks on every pull request.
- Publish approved versioned assets to Exchange and retain the artifact coordinates in the release manifest.
- Deploy the same packaged artifact through Mule Dev, QA, UAT and Production using protected environment approvals.
- Configure API Manager policies, Runtime Manager alerts, log access and post-deployment health checks.

8. CI/CD quality gates

Gate	Salesforce checks	MuleSoft checks	Pass evidence
Pull request	Gearset compare, dependencies, metadata validation, Apex/LWC tests, static analysis	API spec lint, Maven package, MUnit, coverage, dependency/security scan	Both checks pass against linked commits
Integration	Gearset deployment and integration checks	Deploy snapshot to Mule Dev and run contract tests	Job IDs and results linked

Gate	Salesforce checks	MuleSoft checks	Pass evidence
QA	Regression, permissions, negative paths and impacted automation	Contract, transformation, retry, error and performance smoke tests	No open release blocker
UAT	Copado candidate with named business acceptance	Same artifact in Mule UAT with business/API consumer acceptance	Named sign-off for the paired release
Production readiness	Copado validation of immutable commit and approved scope	Protected deployment job for immutable artifact and config review	Release manager approval and rollback readiness
Post-production	Salesforce smoke, monitoring and business validation	Runtime health, API policy, log/alert and downstream checks	Evidence complete before Done

Minimum MuleSoft test expectations

- MUnit covers core success, error and retry paths; coverage thresholds are agreed by the integration lead and enforced in CI.
- API contract tests prove request/response schemas, status codes and backward compatibility for published consumers.
- Secrets and environment properties are injected at deployment and are never stored in source or Jira.
- The pipeline verifies the deployed application reaches a healthy state; skipping deployment verification requires explicit approval and evidence.
- Performance or volume testing is required for high-throughput, batch or latency-sensitive flows.

9. Jira workflow and evidence

The board should be readable by people outside DevOps. Status names describe the work state; transition checks provide the technical control.

Status	Entry condition	Required evidence	Owner
Backlog	Request accepted for triage	Problem, value and affected platforms	Product owner
Ready for Development	Definition of Ready met	Acceptance, architecture, dependencies, test and rollback	Product + architect
In Progress	Linked branch exists	Assignee and repository links	Developer
Code/Peer Review	Pull request open	Diff, checks, risk and reviewer	CODEOWNER
Ready for QA	Both platform CI gates pass	Gearset and MuleSoft CI evidence	Dev lead
In QA	Paired candidate deployed	Salesforce and Mule job IDs	QA lead
Ready for UAT	QA exit met	Results and defect disposition	QA lead
In UAT	Immutable candidate available	Named testers and version manifest	Business owner

Status	Entry condition	Required evidence	Owner
Ready for Release	UAT approved	Sign-off, risk, runbook and rollback	Business + architect
Scheduled	Window and approvals confirmed	Copado release and Mule production job reference	Release manager
Deployed	Both production jobs succeed	Exact job IDs, commits and artifact version	Release manager
Done	Smoke and business checks pass	Final evidence, known issues and sign-off	Product owner

Closure rule
Deployment success is not Done. Done requires verified behavior, complete cross-platform evidence and reconciliation to Git and the release manifest.

10. Security and access model

Control	Implementation requirement	Evidence
Service identities	Separate Gearset, Copado and Anypoint/GitHub identities; no personal production credentials	Owner, purpose and expiry review
Least privilege	Validation identities cannot deploy; production jobs have only required target access	Permission review and test
Secrets	GitHub environments or approved vault; masked output; no secrets in POM, repo, Jira or deployment notes	Secret scan and configuration record
Approvals	Protected branches, Copado release approval and GitHub protected MuleSoft environment	Named approver and timestamp
Network	Review connected apps, callback URLs, IP allowlists and Anypoint control-plane access	Security sign-off
Audit	Retain PR, Gearset, Copado, GitHub Actions and Anypoint logs according to policy	Retrievable pilot evidence

11. Pilot, cutover and rollback

Pilot selection

Choose a change that is small enough to reverse but real enough to exercise the integration. It should include at least one Salesforce metadata change and one MuleSoft contract or transformation change, without billing, authentication or destructive data impact.

Pilot execution

- Run the full story, branch, PR, CI, QA, UAT, release and production path.
- Introduce one safe CI failure and confirm the pipeline blocks promotion.

- Rehearse Salesforce rollback or forward-fix and MuleSoft artifact rollback in non-production.
- Confirm both production deployments reference the approved manifest and no artifact was rebuilt.
- Measure lead time, waiting time, manual touches, failure points and evidence completeness.

Rollback model

Platform	Primary rollback	Decision point
Salesforce	Copado rollback/back-promotion or approved forward-fix from the production tag; reconcile Git immediately	Smoke failure, data risk or release-manager decision
MuleSoft	Redeploy the last known-good immutable artifact and restore approved properties/policies	Health check, API error rate or downstream failure
Coordinated release	Rollback both platforms when the interface contract is incompatible; otherwise isolate only the failed side after architect review	Release manager + architect + business owner

Production cutover checklist

- Release manifest approved and frozen.
- Gearset production validation and Copado Salesforce validation are successful and current.
- MuleSoft artifact is published, tested and promotable without rebuild.
- UAT sign-off, change window, operator, monitoring and rollback owner are confirmed.
- Manual steps are ordered, timed and assigned.
- Post-deployment smoke tests and business contacts are ready.

12. Roles and operating cadence

Activity	Product	Architect	Salesforce	MuleSoft	QA	Release
Refinement and priority	A/R	C	C	C	C	I
Technical design	C	A	R	R	C	I
Build and unit test	I	C	R	R	C	I
CI and review	I	A	R	R	C	I
Integrated QA	I	C	C	C	A/R	I
UAT approval	A/R	C	I	I	C	I
Release readiness	C	C	C	C	C	A/R
Production deployment	I	C	R	R	I	A
Post-deploy verification	A	C	R	R	R	R

R = Responsible, A = Accountable, C = Consulted, I = Informed.

Cadence

Meeting	Purpose	Participants
Weekly implementation review	Remove setup blockers, confirm decisions and track workstream exits	Workstream owners
Daily pipeline review during pilot	Review failed jobs, drift, approvals and cross-platform dependencies	Dev, QA, release
Release readiness review	Confirm scope, evidence, rollback and operators before UAT/Prod	Product, architect, QA, release
Post-release review	Confirm outcome, incidents, metrics and backlog actions	All accountable owners
Monthly control review	Access, bypasses, evidence completeness, drift and tool health	Platform, security, audit

13. Risks, decisions and mitigations

Risk	Impact	Mitigation	Owner
Gearset and Copado both deploy to Production	Conflicting source and incomplete audit trail	Copado-only Salesforce production deployment; Gearset validation-only	Release manager
Copado pipeline mode does not support a required feature	Custom work or delayed rollout	Capability matrix and training pipeline before design approval	Copado owner
MuleSoft production integration is assumed rather than proven	Unsafe or unsupported trigger path	Use protected paired approval first; automate only after proof	Integration lead
Salesforce and MuleSoft releases drift	API contract mismatch	Cross-platform release manifest and integrated QA gate	Architect
Secrets appear in repositories or logs	Security incident	Connected apps, secret store, masking and scans	Security
Permissions are deployed unintentionally	Access regression or overexposure	Explicit Gearset filter and Copado permission policy; persona tests	Salesforce lead
Team cannot retrieve evidence later	Audit and support failure	Evidence schema, retention test and monthly sampling	Delivery lead

Decisions required before build

- Confirm Gearset edition, team ownership and allowed environments.
- Confirm Copado pipeline type, package versions, governance environment and licensing.
- Confirm MuleSoft runtime target, region, business group, API Manager model and deployment identity.

- Approve which tool owns Salesforce UAT; this plan assigns UAT and Production to Copado.
- Approve the pilot story, release window and named accountable owners.

14. Implementation backlog

Draft ID	Story	Owner	Points
IMP-01	Confirm tool editions, pipeline modes and runtime targets	Architecture + tool owners	3
IMP-02	Create GitHub repository controls and release manifest	Platform/DevOps	8
IMP-03	Configure Jira workflow, evidence fields and automation	Jira admin	13
IMP-04	Provision service identities, secrets and access	Security + platform	8
IMP-05	Configure Gearset shared connections and metadata policy	Salesforce lead	8
IMP-06	Implement Gearset PR validation and non-prod CI jobs	Salesforce + DevOps	13
IMP-07	Build Copado training pipeline and user-story/release model	Copado owner	13
IMP-08	Implement Copado UAT, production and back-promotion path	Release + Salesforce	13
IMP-09	Standardize MuleSoft repository, Maven and MUnit controls	MuleSoft lead	8
IMP-10	Implement Anypoint Dev-to-Prod deployment pipeline	MuleSoft + DevOps	13
IMP-11	Implement cross-platform release manifest and evidence links	Architecture + Jira	8
IMP-12	Define integrated QA, security and rollback tests	QA + security	8
IMP-13	Run and accept the cross-platform pilot	Delivery team	13
IMP-14	Publish runbooks, train users and transition to BAU	Delivery lead	8

The draft IDs are planning references, not existing Jira keys. Create the stories under an implementation epic linked to SALDEV-1500 and re-estimate them with the delivery team.

15. Acceptance and handover

Program acceptance criteria

- The same Jira key links the Salesforce and MuleSoft work when both platforms are changed.
- Protected branches reject a failing Salesforce or MuleSoft pull request.
- Gearset validates and deploys Salesforce to lower environments with complete job evidence.
- Copado promotes the approved Salesforce commit through UAT and Production with named approvals.

- The MuleSoft pipeline packages once and promotes the same immutable artifact through Anypoint environments.
- Integrated QA proves the Salesforce-to-MuleSoft contract, security, error behavior and negative paths.
- A rollback rehearsal succeeds in non-production and its decision path is documented.
- Production evidence can be retrieved from Jira without relying on individual inboxes or chat history.
- Runbooks, access owners, support contacts and monthly control measures are accepted by BAU owners.

Handover package

Deliverable	Minimum content
Architecture decision record	Tool boundaries, pipeline mode, environment map and exception decisions
Configuration register	Connections, jobs, filters, branches, credentials, approvals and owners
Runbooks	Normal release, hotfix, rollback, back-promotion, failed job and access recovery
Test pack	CI negative tests, integrated QA, UAT, smoke and rollback evidence
Operating dashboard	Lead time, failed change rate, blocked work, evidence completeness and drift
Training materials	Role-based steps for developers, reviewers, QA, approvers and release managers

Recommended approval
Approve the dual-tool Salesforce model and MuleSoft workstream for a controlled pilot, subject to week-1 confirmation of licenses, Copado pipeline mode, Anypoint runtime target and named owners. Do not authorize production configuration from this document alone.

Appendix A - Definition of Ready and Done

Definition of Ready

- Business outcome, impacted personas and measurable acceptance criteria are clear.
- Salesforce components, MuleSoft APIs/flows, dependencies, data and security impacts are identified.
- Architecture review, change class, release target, test strategy and rollback owner are set.
- Required environments, service identities and upstream prerequisites are available or tracked.
- The story is small enough to review, test and reverse independently where practical.

Definition of Done

- Approved source is merged and Jira, PRs, commits, Gearset jobs, Copado promotion and MuleSoft deployment are linked.
- Automated tests, QA, UAT and production validation passed with exact results and explicit omissions.
- Production used the approved Salesforce commit and MuleSoft artifact without unapproved scope changes.
- Post-deployment Salesforce, API and integration smoke checks passed; known issues are linked and owned.
- Documentation, runbooks, monitoring and release notes are updated.
- Manual changes are reconciled to Git and temporary access or flags are removed or tracked to expiry.

Appendix B - Source notes

The implementation details below were checked against official product documentation accessed on 2 September 2026. Product editions, supported integrations and limitations can change; confirm current contracts and supported features before configuration.

Gearset - Setting up your first CI job: https://docs.gearset.com/en/articles/8333138-setting-up-your-first-ci-job-in-gearset

Gearset - Creating CI jobs in Pipelines: https://docs.gearset.com/en/articles/12460112-creating-ci-jobs-in-gearset-pipelines

Copado - Structure of a User Story: https://docs.copado.com/articles/?_escaped_fragment_=source-format-pipelines-publication/structure-of-a-user-story

Copado - Known limitations in Salesforce Source Format Pipelines: https://docs.copado.com/articles/?_escaped_fragment_=source-format-pipelines-publication/known-limitations-in-salesforce-source-format-pipelines

Copado - Upgrade to Source Format Pipelines: Planning and Recommendations: https://docs.copado.com/articles/?_escaped_fragment_=copado-pipelines-publication/upgrade-to-source-format-pipelines-planning-and-recommendations

MuleSoft - Mule Maven Plugin: https://docs.mulesoft.com/mule-runtime/4.6/mmp-concept

MuleSoft - MUnit Maven coverage: https://docs.mulesoft.com/munit/latest/coverage-maven-concept

MuleSoft - Publishing assets using Maven: https://docs.mulesoft.com/exchange/to-publish-assets-maven

Appendix C - Ready-to-paste stakeholder update

Stakeholder update

A separate implementation plan has been prepared for SALDEV-1500. The proposed delivery model uses Gearset for Salesforce CI and lower-environment validation, Copado for governed Salesforce UAT/Production promotion, and GitHub Actions plus Anypoint for MuleSoft build, MUnit testing, artifact publication and runtime deployment. GitHub remains the source of truth and Jira remains the work, approval and evidence record. The plan includes a 12-week roadmap, role boundaries, security controls, cross-platform quality gates, pilot and rollback steps, risks, acceptance criteria and a Jira-ready implementation backlog. No system configuration or production change was performed as part of the document preparation.